Bachelor of Engineering (B.E.) in
Computer Engineering (BCT)
Electronics, Communication & Information Engineering (BEI)
Third Year Second Part
Elective II Course

Analysis of Algorithms
Chapter 7. NP-Hard and NP-Complete Problems

by:

Er. Bikal Adhikari

(*B.E., M.Sc. Engg., M.Sc. ISE*)

Assistant Professor, Dept. of Electronics & Computer Engg.

Pulchowk Campus, IOE, TU

Chapter Outline

Chapter 7. NP-Hard and NP-Complete Problems

5 hours

7.1 Basic concepts
7.2 Cook's theorem
7.3 NP-Hard graph problems
7.4 NP-Hard scheduling problems
7.5 NP-Hard code generation problems
7.6 Simplified NP-hard problems
7.7 Approximation algorithms: ε- approximation, polynomial time approximation scheme, probabilistically good algorithms
7.8 Vertex cover problem, subset sum problem

Introduction

- The field of complexity theory deals with how fast can one solve a certain types of problems or more generally how much resources does it take: time, memory, space, number of processor etc.
- Up to now we were considering on the problems that can be solved by the algorithms in worst case polynomial time.
- There are many problems and it is not necessary that all the problems have the apparent solution.
- This concept, somehow, can be applied in solving the problem using the computers.
- The computer can solve: some problems in limited time. Eg. Sorting,
- Some problems requires unmanageable amount of time. Eg. Hamiltonian cycles and some problem cannot be solved. Eg. Halting problem
- In this section we concentrate on specific class of problems called NP complete problem.

Tractable and Intractable Problems

- We call problems as tractable or easy , if the problem can be solved using polynomial time algorithms.
- The problems that cannot be solved in polynomial time but requires super polynomial time algorithm are intractable or hard problems.
- There are many problems for which no algorithm with running time better than the exponential time is known.
- Some of them are traveling salesman problem, Hamiltonian cycles, and circuit satisfiability etc.
- Here are some examples of tractable problems (ones with known polynomial-time algorithms):
  - Searching an unordered list
  - Searching an ordered list
  - Sorting a list
  - Multiplication of integers
  - Finding a minimum spanning tree in a graph.

- Here are examples of intractable problems (Ones that have been proven to have no polynomial time algorithm).
- Some of them require a non-polynomial amount of output, so they clearly will take a non-polynomial amount of time.
- Example: *Tower of Hanoi*: We can prove that any algorithm that solves this problem must have the worst case running time that is at least $\mathbf{2^n-1.}$
- List all permutations (all possible ordering) of *n* numbers.

Concept of Polynomial time and Super Polynomial time complexity

- Let's start reminding ourselves of some common functions, ordered by how fast they grow :
  - Constant: $O(1)$
  - Logarithmic: $O(\log n)$
  - Linear: $O(n)$
  - $n \log n$: $O(n \log n)$
  - Quadratic: $O(n^2)$
  - Cubic: $O(n^3)$
  - Exponential: $O(k^n)$
  - Factorial: $O(n!)$

- Computer Scientists divide these functions into two classes:
  - Polynomial running time complexity
  - Super polynomial time complexity

Polynomial Time Complexity

- An algorithm is said to be solvable in polynomial time if the number of steps required to complete the algorithm for a given input is $O(n^k)$ for some non-negative integer k, where n is the complexity of the input.
- Polynomial time algorithms are said to be fast.
- Most familiar mathematical operations such as addition, subtraction, multiplication, and division as well as computing square roots, powers, and logarithms, can be performed in polynomial time.
- Computing the digits of most interesting mathematical constants including π and e, can also be done in polynomial time.
- Example: $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$. $O(n^2)$, $O(n^3)$ etc belongs to polynomial time complexity.

Super Polynomial Time Complexity

- An algorithm whose order-of-magnitude time performance is not bounded from above by a polynomial function of n is called super polynomial time complexity algorithm.
- A function of the form k^n is called super polynomial time complexity algorithm.
- But now some functions which are worse than polynomial but not quite exponential such as $O(n \log n)$, are also (incorrectly) called exponential.
- And some functions which are super worse than exponential, such as the super exponential, Eg:$O(n^n)$ will also be (incorrectly) called exponential.
- A better word than 'exponential' would be '**super-polynomial**'.

Complexity Classes: P, NP, NP-Hard, NP-Complete

Polynomial time algorithms
Linear search- O(n)
Binary Search- O(log n)
Insertion search- $O(n^2)$
Merge Sort- O(n log n)
Matrix multiplication- $O(n^3)$

Exponential time algorithms
0/1 Knapsack – $O(2^n)$
Traveling SP – $O(n^2 2^n)$
Sum of Subsets – $O(2^n)$
Graph Coloring – $O(2^n)$
Hamiltonian Cycle – $O(2^n)$

Complexity Classes: P, NP, NP-Hard, NP-Complete

Class P Problems:

- The class P consists of those problems that can be solved by a deterministic Turing machine in Polynomial time.
- P problems are obviously tractable. More specifically, they are problems that can be solved in $O(n^k)$ for some constant k, where n is the size of the input to the problem.
- Example: Adding two numbers is really easy. Surely, as the number gets larger the computation becomes harder to us human. But to a computer adding large numbers are fairly simple. We can say computers can add two numbers in polynomial time.
- These types of problems which can be solved in polynomial time by computer are known as P problems.

```
Algorithm Nsearch(A,n,key)
{
j=Choice(); ------------O(1)
if(A[j]==key)
{
write(j);
Success();  ------------O(1)
}
write(0);
Failure();  ------------O(1)
}

Total Time Complexity: O(1)
```

NP Class Problems:

- NP is a set of decision problems that can be solved by a Non-deterministic Turing machine in a polynomial time. P is subset of NP (any problem that can be solved by deterministic machine in polynomial time can also be solved by non-deterministic machine in polynomial time.
- The class NP consists of those problems that are verifiable in polynomial time.
- NP is the class of decision problems, for which it is easy to check the correctness of a claimed answer, with the aid of a little extra information. Hence, we aren't asking for a way to find a solution but only to verify that an alleged solution really is correct. Every problem in this class can be solved in exponential time using exhaustive search.
- **Example:** Let's take a little complex problem like prime factorization. We know that every composite number can be expressed as a product of two or more prime factors. Our normal PCs can handle this problem within seconds for numbers up to billion. But as the numbers grow this problem becomes lot harder even for the fastest of computers. So this is not solvable in polynomial time. So factorization is not solvable in polynomial time but the solution is verifiable in polynomial time. These problems are known as NP problems.

$2 = 2$
$3 = 3$
$4 = 2^2$
$5 = 5$
$6 = 2 \cdot 3$
$7 = 7$
$8 = 2^3$
$9 = 3^2$
$10 = 2 \cdot 5$
$11 = 11$
$12 = 2^2 \cdot 3$
$13 = 13$
$14 = 2 \cdot 7$
$15 = 3 \cdot 5$
$16 = 2^4$
$17 = 17$
$18 = 2 \cdot 3^2$
$19 = 19$
$20 = 2^2 \cdot 5$
$21 = 3 \cdot 7$
$22 = 2 \cdot 11$
$23 = 23$
$24 = 2^3 \cdot 3$
$25 = 5^2$
$26 = 2 \cdot 13$
$27 = 3^3$
$28 = 2^2 \cdot 7$
$29 = 29$
$30 = 2 \cdot 3 \cdot 5$
$31 = 31$
$32 = 2^5$
$33 = 3 \cdot 11$
$34 = 2 \cdot 17$
$35 = 5 \cdot 7$
$36 = 2^2 \cdot 3^2$
$37 = 37$
$38 = 2 \cdot 19$
$39 = 3 \cdot 13$
$40 = 2^3 \cdot 5$
$41 = 41$
$42 = 2 \cdot 3 \cdot 7$
$43 = 43$
$44 = 2^2 \cdot 11$
$45 = 3^2 \cdot 5$
$46 = 2 \cdot 23$
$47 = 47$
$48 = 2^4 \cdot 3$
$49 = 7^2$
$50 = 2 \cdot 5^2$
$51 = 3 \cdot 17$

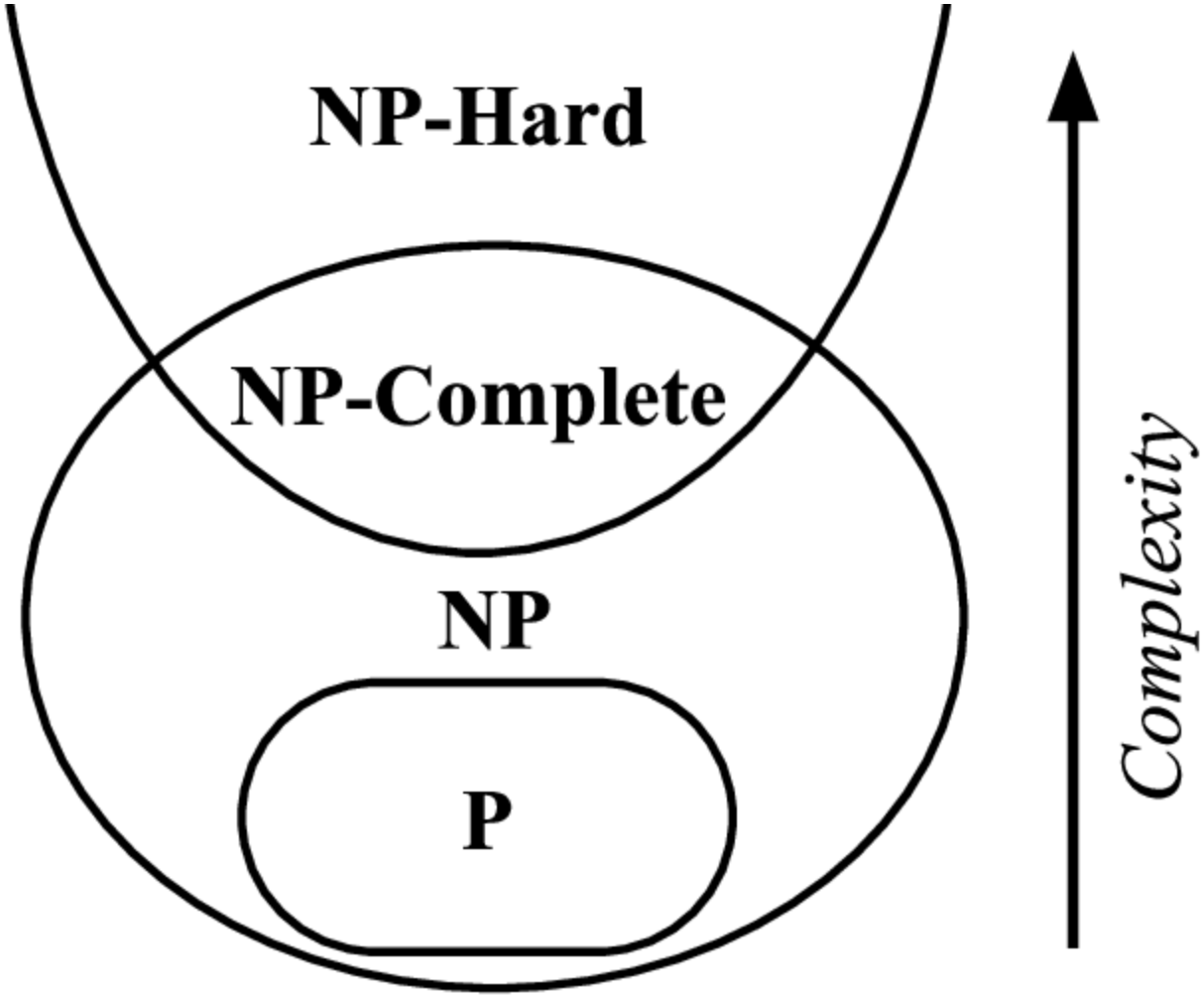
NP Complete:

- NP-Complete problem is a complexity class which represents the set of all problems *X* in NP for which it is possible to reduce any other NP problem Y to X in polynomial time.
- NP-Complete problems are the hardest problems in NP set. A decision problem *L* is a NP-Complete if:
  - L is in NP(Any given solution for NP-Complete problems can be verified quickly but there is no efficient known solution.)
  - Every problem in NP is reducible to L in polynomial time.
- Intuitively this means we can solve Y quickly if we know how to solve X quickly. Precisely Y is reducible to X, if there is a polynomial time algorithm f to transform instances y of Y to instances x = f(y) of X in polynomial time, with the property that the answer to y is yes, if and only if the answer to f(y) is yes.
- Examples: An interesting example is the graph isomorphism problem, the graph theory problem of determining whether a graph isomorphism exists between two graphs. Two graphs are isomorphic if one can transformed into other simply by renaming vertices.

- Consider these problems:
  - Graph Isomorphism: Is graph G1 isomorphic to graph G2?
  - Sub-graph Isomorphism: Is graph G1 isomorphic to sub graph of G2?
- The easiest way to prove that some new problem is NP-Complete is first to prove that it is in NP, and then to reduce some known NP-complete problems. The list below contains some well-known problems that are NP-Complete when expressed as decision problem.
  - Boolean Satisfiability Problem(SAT)
  - Knapsack Problem
  - Hamiltonian Path Problem
  - Traveling Salesman Problem
  - Sub-graph isomorphism problem
  - Subset sum problem
  - Clique problem
  - Vertex cover problem
  - Independent set problem
  - Dominating set problem

NP hard:

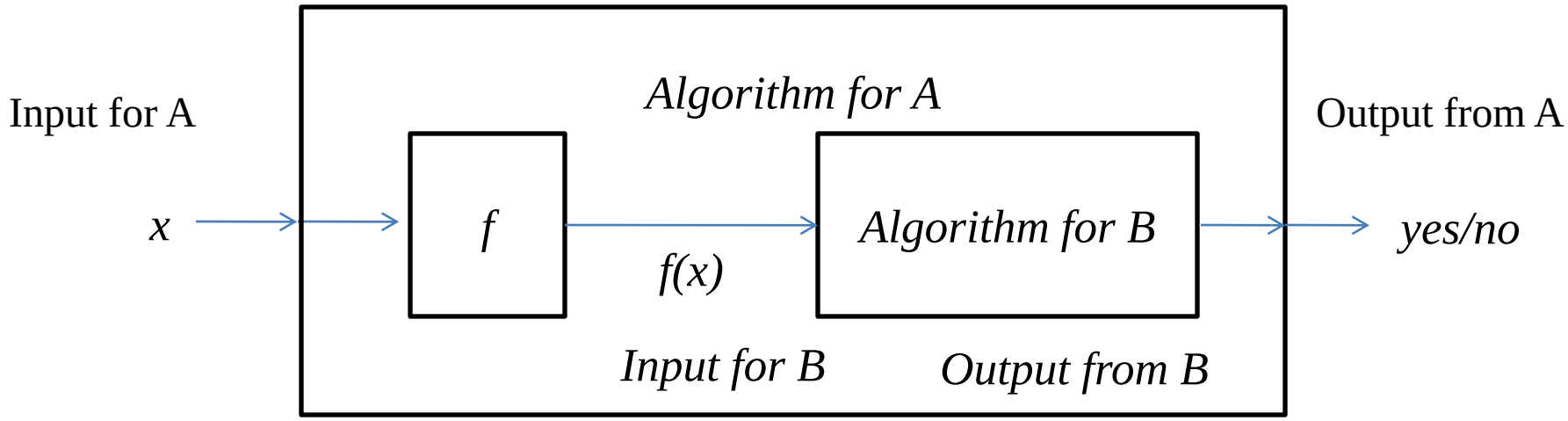
- A problem is NP-Hard if every NP problem can be reduced to it in polynomial time.
- Intuitively, these are the problems that are at least as hard as the NP complete problems.
- The precise definition here is that a problem X is NP-hard, if there is an NP complete problem Y, such that Y is reducible to X in polynomial time.
- But since any NP complete problem can be reduced to any other NP complete problems in polynomial time.
- All NP complete problem can be reduced to a NP hard problem in polynomial time.
- If there is a solution to one NP hard problem in polynomial time, then there is a solution to all NP problems in polynomial time.
- Example: The halting problem is NP hard problem. This is the problem that given a program P and input I, will it halt? This is a decision problem but it is not in NP. It is clear that any NP-complete problem can be reduced to this one. As another example, any NP problem is NP-hard.


NP-Hard
NP-Complete
NP
P
Complexity

	P	NP	NP-complete	NP-hard
Solvable in polynomial time	✓			
Solution verifiable in polynomial time	✓	✓	✓	
Reduces any NP problem in polynomial time			✓	✓

Polynomial Time Reduction

- Given two decision problems A and B, a polynomial time reduction from A to B is a polynomial time function f that transforms the instances of A into instances of B such that the output of algorithm for the problem A on input instance x must be same as the output of the algorithm for the problem B on the input instance f(x) as shown in the figure below.
- If there is polynomial time computable function f such that it is possible to reduce A to B, then it is denoted as $A \leq pB$.
- The function f described above is called reduction function and the algorithm for computing f is called reduction algorithm.



Cooks Theorem

- In the computational complexity theory, the Cook-Levin theorem, also known as Cook's theorem states that the Boolean Satisfiability problem is NP-Complete.
- That is any problem in NP can be reduced in polynomial time by a deterministic Turing machine to the problem of determining whether a Boolean formula is satisfiable.
- An important consequence of this theorem is that if there exists a deterministic polynomial time algorithm for solving Boolean satisfiability, then every NP problem can be solved by deterministic polynomial time algorithm.
- The question of whether such an algorithm for Boolean satisfiability exists is thus equivalent to P versus NP problem, which is widely considered the most unsolved problem in theoretical computer science.

Theorem

- The satisfiability problem is NP Complete.
- What is SAT?
- Boolean satisfiability or simply SAT is the problem of determining if a Boolean formula is satisfiable or unsatisfiable.
- Satisfiable: If the Boolean variables can be assigned values such that the formula turns out to be TRUE, then we say that the formula is satisfiable.
- Unsatisfiable: If it is not possible to assign such values, then we say that the formula is unsatisfiable.
- Example:

$$F = A \wedge \overline{B} \text{ is satisfiable because}$$

$$\text{A} = \text{TRUE and B} = \text{FALSE makes F} = \text{TRUE}$$

$$G = A \wedge \overline{A} \text{ is unsatisfiable because G is always false}$$

Approximation Algorithms

Concept:

- An approximate algorithm is a way of approaching NP-Completeness for the optimization problem.
- This technique does not guarantee the best solution.
- The goal of an approximation algorithm is to come as close as possible to the optimum value in a reasonable amount of time which is at most the polynomial time.
- Such algorithms are called approximation algorithm or heuristic algorithm.
- **Example:** For the traveling salesperson problem, the optimization problem is to find the shortest cycle, and the approximation problem is to find a short cycle, for the vertex cover problem the optimization problem is to find the vertex cover with fewest vertices and the approximation problem is to find the vertex cover with few vertices.

Types of Approximation

- **ε-approximation:** An ε-approximation algorithm is one that guarantees a solution within a factor of (1+ε) of the optimal, for any chosen ε>0. The parameter ε allows the user to control the trade-off between accuracy and computational complexity: smaller values of ε yield solutions closer to optimal but may require more time or resources. These algorithms are particularly useful in geometric optimization and clustering problems, where exact solutions are computationally expensive, yet near-optimal solutions are sufficient for practical purposes

- **A Polynomial Time Approximation Scheme:** A Polynomial Time Approximation Scheme (PTAS) is a family of algorithms that, for any fixed ε>0, produces a solution within (1+ε) of the optimal in polynomial time. The defining characteristic of PTAS is its flexibility: the user can specify the desired accuracy, and the scheme adapts accordingly. While the running time is polynomial in the input size, it may grow rapidly as ε decreases, making extremely small values impractical. PTAS is a powerful framework for problems such as Knapsack and scheduling, where exact solutions are NP-hard but arbitrarily close approximations are achievable.

Types of Approximation

- **Probabilistically good algorithms:** Probabilistically good algorithms are randomized methods that provide near-optimal solutions with high probability or in expectation. Unlike deterministic approximation algorithms, their guarantees are probabilistic, meaning that while a single run may not always yield the best result, the expected performance over multiple runs is provably close to optimal. These algorithms often exploit randomness to simplify complex optimization tasks, and they are widely applied in problems such as Set Cover, where randomized rounding of linear programming relaxations yields efficient and effective solutions. Their strength lies in balancing simplicity, speed, and strong expected performance.

Other Topics

- ✓ **NP Hard-Graph Problems:**
  - ❑ Clique Decision Problem
  - ❑ Node Cover Decision Problem
  - ❑ Chromatic Number Decision Problem
  - ❑ Directed Hamiltonian Cycle
  - ❑ Travelling Salesperson Decision Problem
  - ❑ AND/OR Graph Decision Problem
- ✓ **NP-Hard Scheduling Problems:**
  - ❑ Scheduling Identical Processors
  - ❑ Flow Shop Scheduling
  - ❑ Job Shop Scheduling
- ✓ **NP-Hard Code Generation Problems:**
  - ❑ Code Generation with Common Subexpressions
  - ❑ Implementing Parallel Assignment Instructions
- ✓ **Some Simplified NP-Hard Problems**

Refer to Computer Algorithms by Ellis Horowitz, Sartaj Sahni and Sanguthevar Rajasekaran for these topics

The Vertex Cover Problem

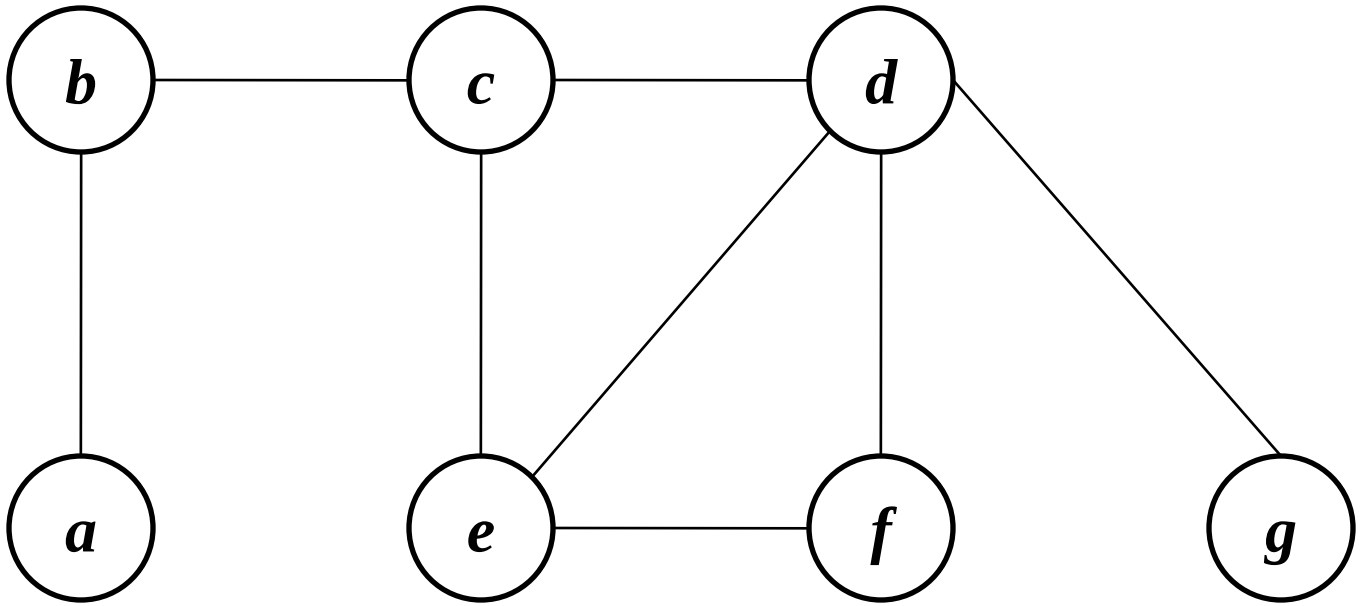
- Problem statement
  - a vertex cover of an undirected graph is a set of vertices $V' \subseteq V$ such that if $(u,v) \in G$ then either $u \in V'$ or $v \in V'$ or both
  - the vertex cover problem is the find the smallest subset that covers an undirected graph
- The following approximation algorithm has a ratio bound of 2

```
APPROX-VERTEX-COVER(G)
1  C ← ∅
2  E' ← E[G]
3  while E' ≠ ∅
4       do let (u, v) be an arbitrary edge of E'
5          C ← C ∪ {u, v}
6          remove from E' every edge incident on either u or v
7  return C
```

Q. Find the minimum vertices covered by using vertex cover problem

Solution:

E = {(a,b), (b,c), (c,d),(c,e), (d,e), (d,f), (e,f), (d,g)}

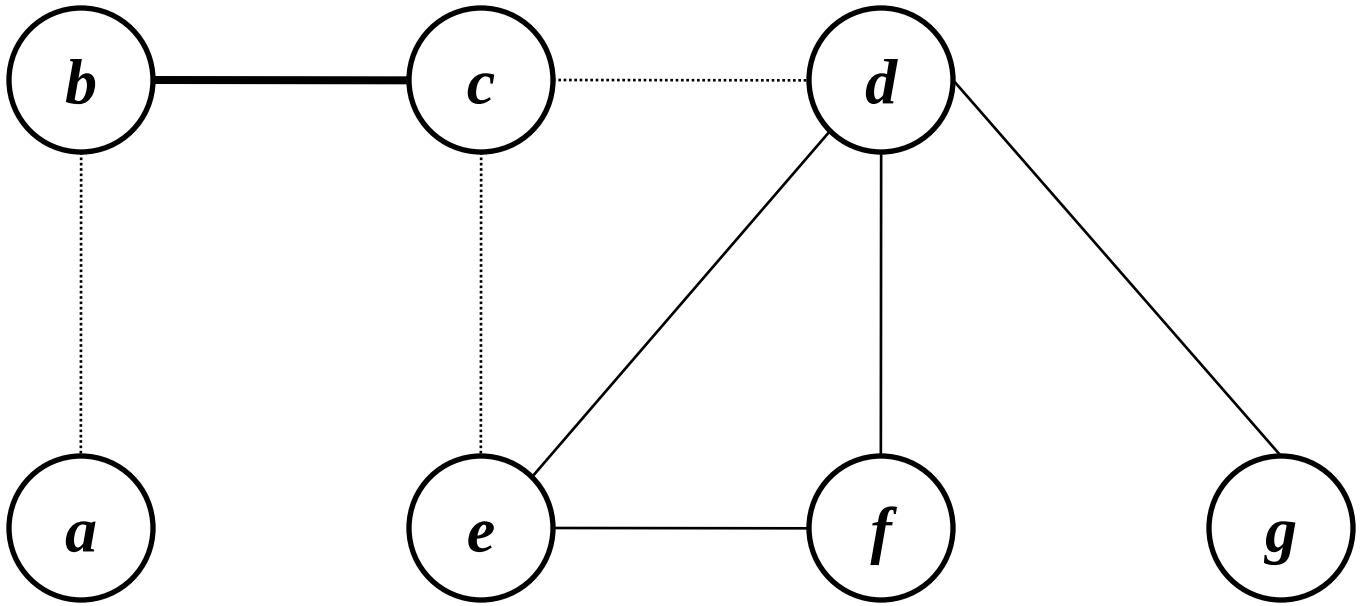


Step 1: Lets choose the edge (b,c)

Eliminate edges incident to the vertex b and c.

E = {(b,c), (d,e), (d,f), (e,f), (d,g)}

C = {(b,c)}

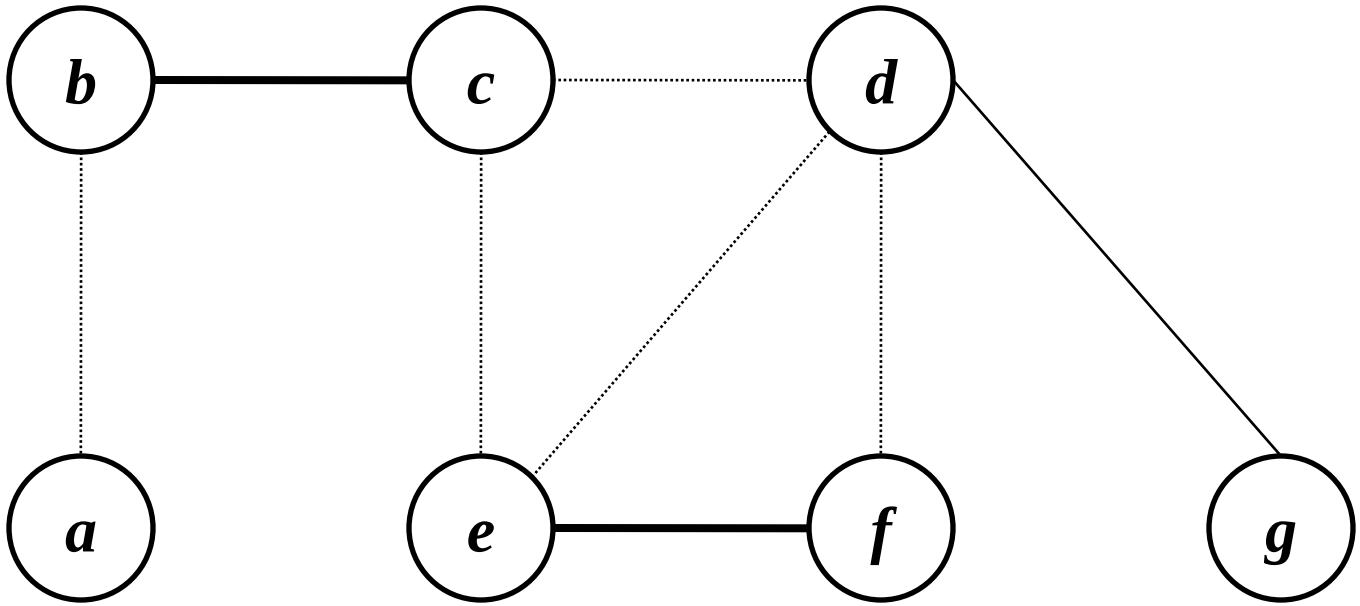


Step 2: Lets choose the edge (e,f)

Eliminate edges incident to the vertex e and f.

E = {(d,g)}

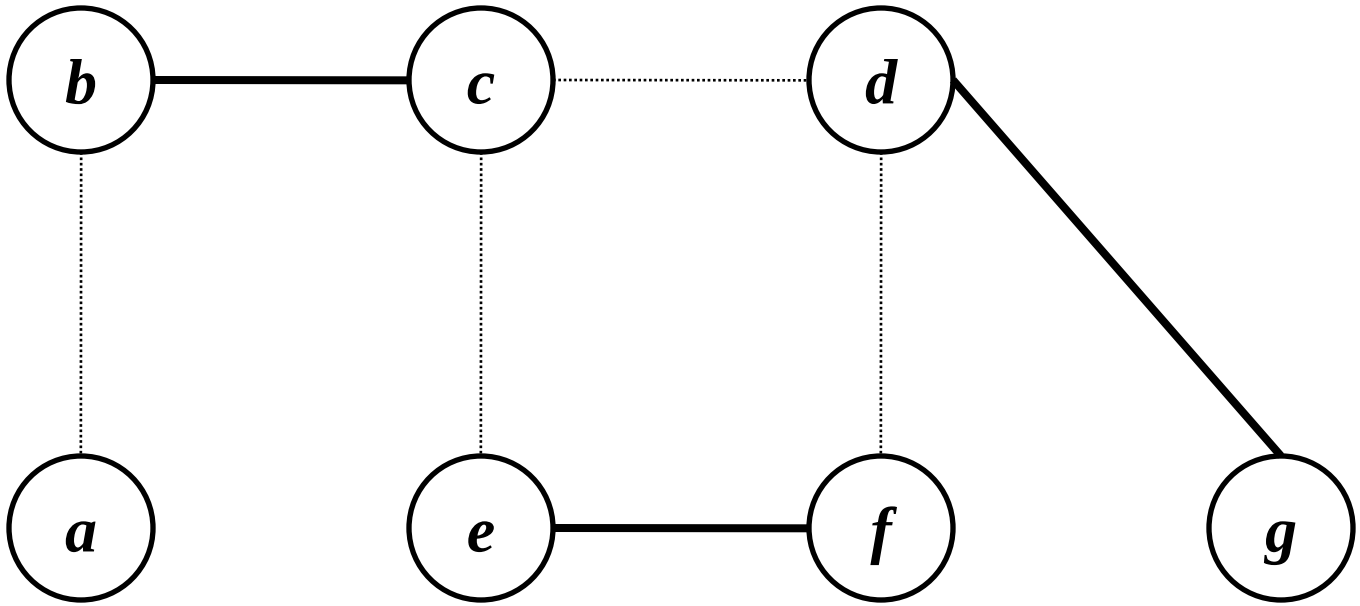
C = {(b,c), (e,f)}



Step 3: Lets choose the edge (d,g)

Eliminate edges incident to the vertex d and g.

E = Φ

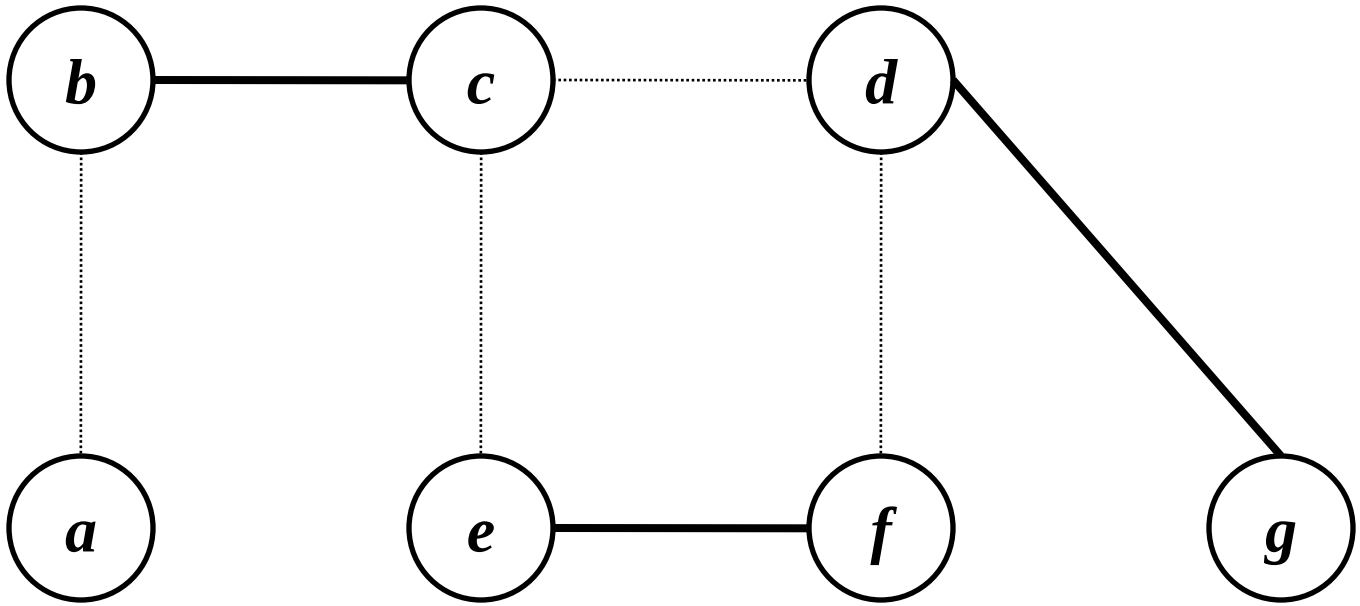


Step 3: Lets choose the edge (d,g)

Eliminate edges incident to the vertex d and g.

E = Φ

C = {(b,c), (e,f),(d,g)}



How it works and the complexity

- How the algorithm works
  - C contains the cover which is initialized to ϕ
  - E' is a copy of the edges of the graph
  - every time an edge (u,v) is selected from E', it is added to the cover and edge incident on u or v is removed from E'; the algorithm stops when E' is ϕ
- The running time is O(E)
- The ratio bound can be proved to be 2

Thank You!

Any Queries?